AI ENGINEER STUDY GUIDE · #04

Finetuning & Local Models

When to change the weights, how to do it cheaply, and how to run the result on your own machine.

1. Finetuning fundamentals — the prompt → RAG → finetune ladder
2. LoRA & QLoRA — low-rank adaptation and 4-bit training
3. PEFT & Mixture of Experts — the family map and a contrast
4. Local models & Ollama — quantization, GGUF, serving

Practical deep understanding · build, break, reason from first principles

SECTION 1

Finetuning Fundamentals

Finetuning changes the model's **weights** to bake in a behavior. Prompting and RAG change the model's **input** to steer behavior it already has. The first real skill is telling those apart — because most problems people reach for finetuning to solve are input problems.

The one distinction everything hangs on

A model is frozen **weights** (its baked-in knowledge and instincts) plus the **context** you hand it at runtime. To change its behavior you either change the context (prompting, few-shot, RAG — same brain, better question) or change the weights (finetuning — rewire the brain so the behavior is permanent). Almost every "should I finetune?" question collapses to: *is this a context problem or a weights problem?*

Mental model — the analogy

Prompting is **giving instructions**. RAG is **handing over the reference binder**. Finetuning is **sending someone to training** so the skill becomes second nature. You don't send someone to a six-week course to answer something they could have looked up.

What finetuning is good and bad at

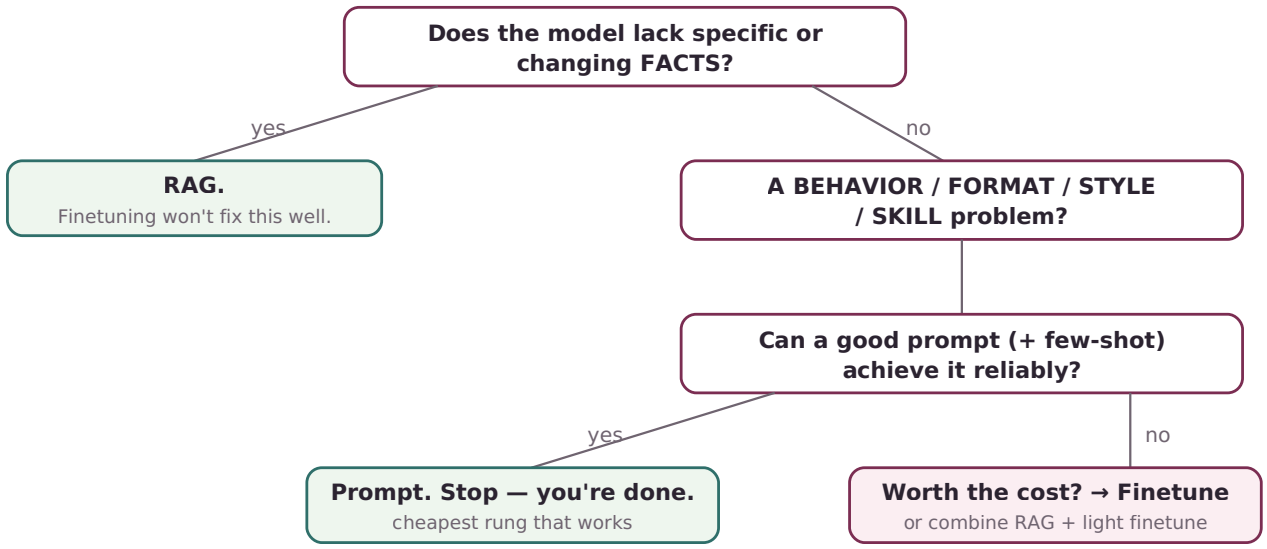
Finetuning is a **nudge**, not a rebuild: you take a model that already knows language and reasoning and tilt it toward a narrow behavior, which is why it needs far less data than pretraining. It is genuinely good at **form and format**, **behavior and tone**, **narrow skills** with lots of labeled examples, and **compressing** a long prompt into the weights. It is bad — and people get this wrong — at **adding fresh knowledge** and **keeping knowledge current**. Finetuning tends to teach the *shape* of your examples, not reliably memorize their *content*, and frozen weights can't track anything that changes.

Mental model — skill vs. knowledge

Finetuning teaches a **skill or a style**. RAG supplies **knowledge**. If what you need is “know this fact,” you almost never want finetuning.

The decision: prompt → RAG → finetune

A deliberate ladder, cheapest first. Climb only when the rung below genuinely fails. Start with a clear prompt and few-shot examples; reach for RAG when the problem is missing *facts*; reach for finetuning only when the need is about *form/skill* **and** prompting can't get there reliably (or the working prompt is so long it's worth baking in).



Climb the ladder only when the rung below genuinely fails.

The bottom branch matters: **RAG and finetuning are not rivals.** Strong systems often do both — RAG supplies current grounded facts, a light finetune locks in format and behavior. They solve different problems and stack cleanly.

Limitation — failure modes to respect

Catastrophic forgetting: aggressive narrow finetuning can erase general ability. **Teaching the shape, not the substance:** small datasets produce confident, well-formatted nonsense. **Data quality dominates:** a few hundred clean examples beat thousands of noisy ones — garbage gets baked in permanently. **Distribution mismatch:** the model degrades on inputs that don't look like its training examples.

Project — the Filing Analyst Copilot

The facts (what a 10-K says) → RAG, always — specific, per-company, changing. **The behavior** (always cite, answer in house format, refuse when context doesn't support an answer) → form and skill, a legitimate finetune candidate. The module project finetunes *behavior* on top of a RAG system that still does all the *knowledge* work — the ladder working as designed.

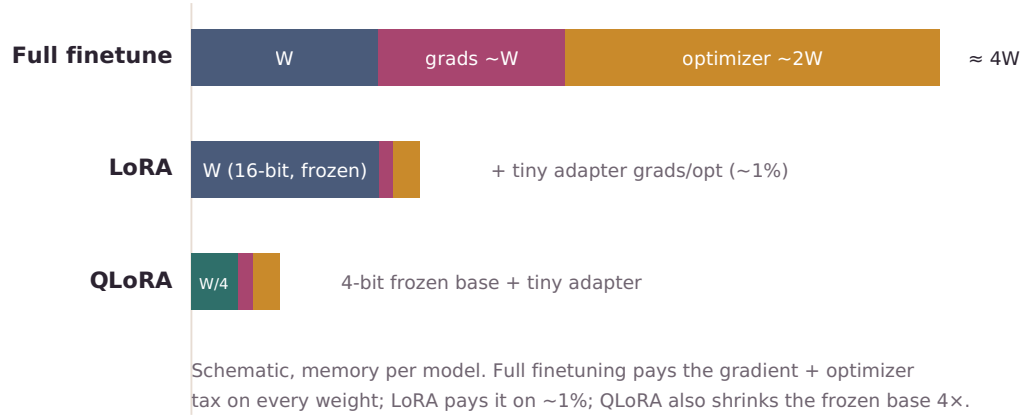
SECTION 2

LoRA & QLoRA

Full finetuning is expensive because **training** a weight costs far more memory than **storing** it. LoRA freezes the original weights and learns a tiny low-rank side path instead — ~1% of the parameters. QLoRA also crushes the frozen base to 4-bit so the whole thing fits on hardware you have.

Why full finetuning is so expensive

Storing a model is cheap; training it is not. GPU memory during training holds four things: the **weights** (~W), the **gradients** (~W), the **optimizer state** (Adam keeps two numbers per weight, ~2W), and activations. That's roughly **4W** before activations — a 7B model at ~14GB needs ~56GB+ to fully finetune.



Schematic, memory per model. Full finetuning pays the gradient + optimizer tax on every weight; LoRA pays it on ~1%; QLoRA also shrinks the frozen base 4×.

The key realization: gradients and optimizer state — three-quarters of the cost — only exist for weights you actually *train*. Freeze a weight and it costs storage and nothing else. So freeze almost everything and train a tiny add-on. That's LoRA.

The low-rank idea

Finetuning's result is `W_finetuned = W + ΔW` , where ΔW is the total change. LoRA's insight, from the paper: that change has **low intrinsic rank** — it's far simpler than its size suggests and can be reconstructed from much less information, like a smooth gradient described by a few numbers instead of every pixel. So factor it into two skinny matrices whose product reconstructs it.

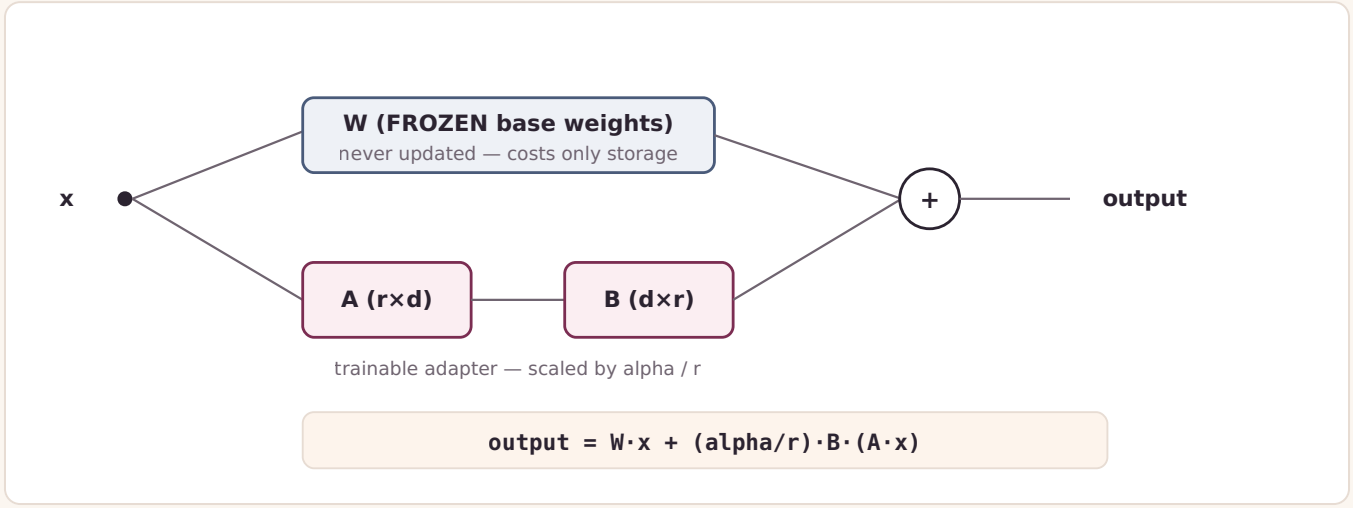
```
ΔW  (d × d)   ≈   B (d × r)  ×  A (r × d)        r is TINY (8, 16, 32)

d = 4096, r = 8:
  ΔW  full :  4096 × 4096  = 16,777,216 numbers
  B + A    :  2 × (4096×8) =     65,536 numbers   →  ~0.4% of ΔW
```

r (rank) is the one capacity knob; 8–16 is usually plenty. You train only those two skinny matrices, so the expensive gradient + optimizer cost is now ~0.4% of what it was.

Frozen base, trainable side path

LoRA leaves W **frozen** and adds the low-rank product as a parallel path.



alpha scales the adapter's contribution by `alpha/r` (a common convention is `alpha = 2r`) — think "how loud the adapter is." At init, A is random and **B is zero**, so the model starts *exactly* as the base and only departs as far as the data demands — which is why LoRA is stable and resists forgetting.

Mental model — two consequences

Adapters are tiny and swappable (a few MB of A/B matrices — keep one base, hot-swap tasks). **You can merge for deployment**: `W + (alpha/r)·B·A` is just arithmetic, foldable into a normal standalone model with zero runtime overhead. You'll merge before serving via Ollama.

QLoRA — fitting it on hardware you have

LoRA shrinks training cost, but you still hold the frozen base to run the forward pass (~14GB for 7B). QLoRA stores that frozen base in **4-bit** (~4× smaller, ~3.5GB) via three tricks: **NF4** (a 4-bit format with values placed where bell-curved weights actually cluster — fine near zero, coarse in the tails); **double quantization** (quantize the quantization constants too); and **paged optimizers** (spill optimizer state to CPU on memory spikes instead of crashing).

Mental model — LoRA vs. QLoRA

LoRA freezes the model and trains a tiny side path, so you stop paying the 3× gradient-and-optimizer tax on the whole thing. QLoRA additionally crushes the frozen part to 4-bit so even *holding* it is cheap. **LoRA makes training affordable; QLoRA makes it fit.**

Papers

LoRA (Hu et al., 2021) — the low-rank-ΔW hypothesis and frozen-base side path. **QLoRA** (Dettmers et al., 2023) — NF4, double quantization, paged optimizers. Read LoRA first.

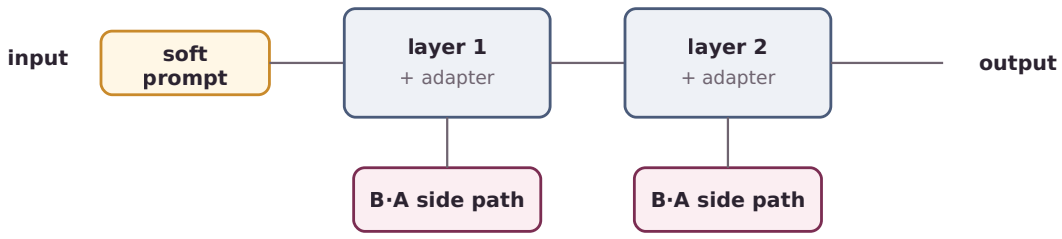
SECTION 3

PEFT & Mixture of Experts

PEFT is the family of “train a tiny slice, freeze the rest” methods — LoRA is one sibling. Mixture of Experts is a different idea: a way to build a bigger, cheaper-to-run *base* model. PEFT makes **training** cheap; MoE makes **capacity** cheap. Same module, different questions.

PEFT: where you put the small trainable piece

Every PEFT method shares one premise — a pretrained model already knows almost everything, so adapting it is a *small* change you shouldn't pay to retrain in full. They differ in **where** the trainable piece goes: at the **input** (prefix/prompt tuning — freeze everything, learn “soft prompts,” vectors gradient descent chooses in the model's own language); **inside** (adapters — insert small trainable layers; not mergeable, so they add inference latency); or **alongside** (LoRA — reparameterize the weight change; mergeable to zero cost).



Three places to put the trainable piece:

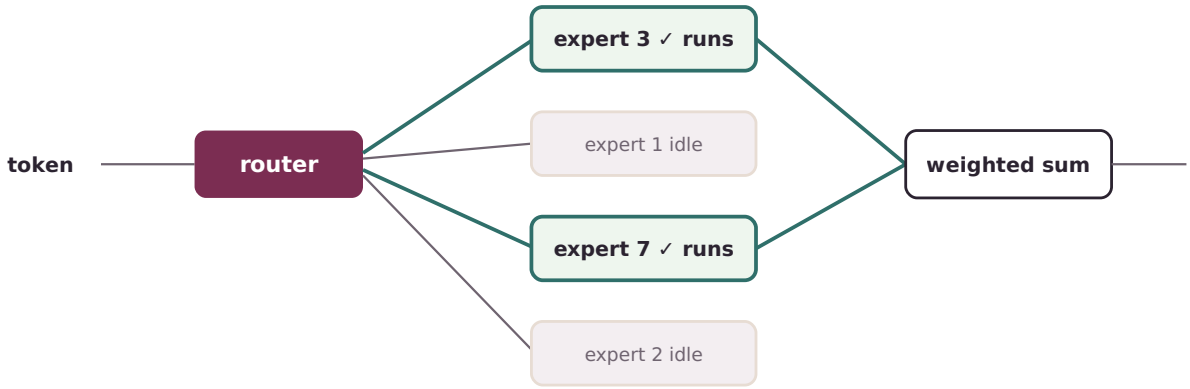
- input → prefix / prompt tuning (freeze all, steer input)
- inside → adapters (insert small trainable layers; not mergeable
- alongside → LoRA (low-rank side path; merges to zero cost)

Method	What's trained	Inference cost added	Notes
Full finetune	all weights	none	baseline; brutal to train
Adapters	inserted small layers	small (always)	first PEFT method; not mergeable
LoRA / QLoRA	low-rank side path	**none after merge**	the practical default
Prefix tuning	per-layer input vectors	small	model fully frozen
Prompt tuning	input-layer vectors	tiny	fewest params; needs a big base

LoRA dominates because it's the only one that's *both* tiny to train *and* free at inference after merging, without inserting anything into the forward pass.

Mixture of Experts — a different question

MoE is not a finetuning method. A dense model runs *every* parameter for *every* token. MoE replaces one big feed-forward block with many smaller **experts** plus a **router** that picks just a few (often top-2) per token; the rest sit idle.



N experts exist (large total capacity) — only top-k run per token (small compute). Cost is paid in memory: all experts stay loaded.

This splits two things that used to be welded together: **total parameters** (capacity — all experts, large) and **active parameters** (compute per token — only the few that ran, small). Mixtral 8×7B has the capacity of ~47B while doing the compute of ~13B per token — big-model quality at small-model speed, **paid for in memory** since all experts must be loaded.

Mental model — the dispatcher

A dense model is one generalist handling every token. MoE is a large team of specialists with a dispatcher (the router) handing each token to the two most relevant people. Total expertise is huge; each token pays for two people's time. The dispatcher's hard job — **load balancing** — is not overloading the same two stars every time (solved with an auxiliary balancing loss; Switch Transformer simplifies routing to top-1).

Why it's here, and where it stops

You will **not** train an MoE — that's pretraining-scale. Carry away: MoE explains why some open models punch above their inference cost; it sharpens the PEFT contrast (cheap *adaptation* vs. cheap *capacity*); and dense-vs-MoE tells you a local model's **memory** demand (MoE: all experts loaded) vs. its **speed** — directly relevant to a Mac's ceiling.

Papers

PEFT survey (Lialin et al., *Scaling Down to Scale Up*) — the family map. **Switch Transformer** (Fedus et al., 2021) — clearest MoE read. **Mixtral** (Jiang et al., 2024) — the modern open MoE example.

SECTION 4

Local Models & Ollama

> Running locally trades cloud horsepower for privacy, cost, and control — which only works if you shrink the model to fit your machine. Quantization shrinks it; GGUF packages it; Ollama serves it. This is where your finetuned adapter stops being a Colab artifact and runs on your Mac.

Quantization: the lever that makes local possible

Store each weight in fewer bits. 16-bit is normal; 8-bit halves memory; 4-bit quarters it. Models tolerate this well down to 4-bit; below that, quality drops off noticeably.

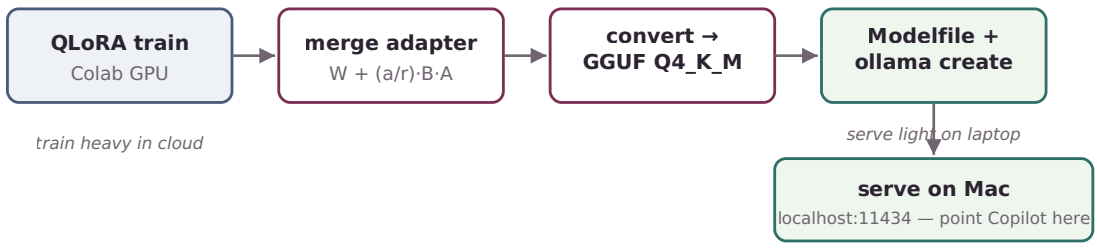
```
APPROX MEMORY TO RUN  ≈  P billion × (bits ÷ 8) GB

7B model:   16-bit → ~14 GB (won't fit)    8-bit → ~7 GB (tight)
            4-bit  → ~3.5 GB (fits comfortably)
```

Labels decode simply: the **number** is bits per weight (Q4/Q5/Q8); **_K** is a smarter “k-quant” that spends bits unevenly; **_S/_M/_L** is a size-vs-quality dial. **Q4_K_M** is the go-to default — best balance of size, speed, and quality.

GGUF and Ollama

GGUF is a single-file container for local inference (CPU and Apple Silicon included): quantized weights, architecture, tokenizer, and prompt template all baked in — portable, no Python framework needed. **Ollama** wraps the inference engine so local models feel like an API: it pulls models from a registry, serves them on `localhost:11434` in a cloud-API-like shape, and manages loading/unloading.



The Modelfile — your finetune becomes an Ollama model

A Modelfile is the recipe (Ollama's Dockerfile) that assembles a custom model — including from your own GGUF.

```
# Modelfile
FROM ./filing-copilot-merged-Q4_K_M.gguf

SYSTEM """You answer questions about SEC filings. Every claim must
cite the filing section it came from. If the retrieved context does
not support an answer, say so plainly rather than guessing."""

PARAMETER temperature 0.2
PARAMETER num_ctx 8192
```

```
ollama create filing-copilot -f Modelfile   # register your custom model
ollama run filing-copilot                    # use it like any other
```

Info — the SYSTEM prompt caveat

`SYSTEM` bakes in default behavior that travels with the model — but (lesson from the agents module) it should drive **behavior**, not **self-certify** groundedness. Your actual citation-checking still lives in application logic, not here.

Limitation — the Mac memory reality

A 4-bit **small model (~3B, 2–4GB)** runs comfortably — your serving target. A 4-bit **7–8B** runs with less headroom. Larger swaps painfully or won't load. **MoE models are deceptive:** small active compute but *all experts must be in memory*, so they need far more RAM than their speed suggests.

Project — closing the loop

Train heavy in Colab, serve light on the Mac. Merge the QLoRA adapter → convert to Q4_K_M GGUF → write a Modelfile with a citation-enforcing SYSTEM prompt and low temperature → `ollama create` → point the Filing Copilot at `localhost:11434` → run the before/after comparison on the held-out eval set. The complete circuit: when to finetune, how LoRA/QLoRA make it affordable, where it sits among PEFT methods, and how to run it locally.